

P?S Extended PDP-8 Simulator (PEPS) Package STRIP.TEC Macro Internal Description

Copyright © 1975-1989, Charles Lasner Associates
Copyright © 1990-2018, »CLA Systems
ClaSystems@gmail.com

Last edit: 26-Aug-2018 cjl

Technical level: Intermediate+. Some working knowledge of TECO required; access to a TECOC reference document is highly recommended to fully understand some of the more arcane TECO commands used. (Note: Advanced TECO users may use a larger subset of TECOC in their manual editing work; however, some of the commands documented here will exceed even their working knowledge of TECO as well; certain commands are hardly ever used outside of TECO macros; some are virtually exclusive to TECOC.)

Introduction.

Various components of the P?S Extended PDP-8 Simulator (PEPS) package (an extension to the SIMH PDP-8 simulator) for windows use the STRIP.TEC macro file to filter and correct various file quirks in output files created within the SIMH simulated PDP-8 environment. These aberrations are usually caused by minor programming errors that went virtually unnoticed when originally used on then-present day hardware. This is because physical lineprinters and similar output devices generally could tolerate modest formatting errors (and many programmers were complacent).

Prior to the use of simulators, printer users were willing to tear off output perhaps after manually separating pages (or at least removing extraneous pages on either side of the listing). Few programmers cared about minor anomalies; merely obtaining a listing was often a luxury.

Note: Some users were especially picky regarding the way fan-folded listings were folded, preferring a top page with the fold at the bottom; as necessary, this could include an additional blank page after the last printed page with the fold at the top. Failing to obtain this precise result led to manually refolding the pages. Repeat printing the listing merely to obtain the desired result was considered prohibitive. Savvy users learned the behavior of specific lineprinters to generally assure the final result would be obtained without resorting to refolding; however, if the only "problem" was the "wrong" fold direction, listings were generally manually refolded while waiting for the next file to print; access to lineprinters was typically so limited that elaborate scenarios were planned to obtain printed output sometimes from relatively remote computer installations.

Processing output files.

Today we expect to either read the lineprinter-oriented output directly in an editor such as windows Notepad, or perhaps publish the output through a word-processor or a .PDF file creator. As such, all minor aberrations must first be removed for the benefit of these "modern" facilities, which tend to easily reveal such quirks (which are often dealt with in a heavy-handed manner). (Generally, fan-fold paper is no longer used, but may be simulated using available utilities.)

The burden of printing has shifted to post-processing within the host environment (in this instance windows or perhaps MS-DOS) which can be somewhat "hostile" to the raw output created by prevailing programs such as OS/8 PAL8 and P?S/8 PAL. In certain cases, the output can be tainted by the deliberate insertion of inappropriate characters as part of an ill-conceived design (which was demanded by programming managers with a twisted agenda). The reasoning behind all of the problems dealt with here is beyond the scope of this document; the only concern is how STRIP.TEC resolves them.

Note: For more information about the problems addressed above, contact the author using the indicated email address and reference the "Typeset-8 Edict".

The PEPS Package enhancements allow easy review of the simulated paper-tape and lineprinter output files. For those interested in viewing specific quirks, the files can be displayed before and after STRIP.TEC is applied.

Note: Viewing OS/8-generated text files before filtering may prove difficult; the files are usually created with mark parity set on every character. Editors are available such as EditPad Lite, which can interpret text encoding as seven-bit ASCII using nonstandard settings.

TECO macro arguments and execution.

The TECO macro STRIP.TEC is launched by the following command inside several batch files which are initiated by the shortcuts (Commands) section of the package:

```
TECOC MUNG STRIP.TEC,%STRIPFILE%
```

The STRIPFILE environment variable contents are derived from the %1 argument as passed to the relevant batch files. By specifying an independent variable, flexibility in command formation is achieved and readability is enhanced. Generally the contents of STRIPFILE will be the complete pathname of the file to be processed.

The MUNG sub-command places the entire contents of STRIP.TEC into an internal buffer within TECOC.

Note: Various implementations of TECO have equivalent startup mechanisms which may not be compatible with TECOC. The differences, although generally minor, must be dealt with as required. STRIP.TEC is designed to work with only TECOC; minor changes may be necessary to use STRIP.TEC with other TECO implementations.

As used here, the contents of the STRIPFILE environment variable are placed into the edit buffer usually associated with text while performing conventional editing. The execution of the STRIP.TEC macro starts with the entire macro contents read into another internal buffer (which can be envisioned as an internal unnamed Q-register). Since the STRIP.TEC macro is designed to take the pathname of the passed file as an argument, it will process the text as necessary to create read or write commands to the passed file as required.

Note: Startup conventions and exact mechanisms used to pass command-line arguments to TECO are extremely implementation-specific. In some versions, the string is passed as the text within a Q-register perhaps different from another Q-register within which are the macro contents; see specific TECO release documentation for more information regarding startup and initial argument passing considerations.

Except where otherwise indicated, \$ refers to the escape character (<ESC>) (the ASCII character code for <ESC> is 27 decimal, 32 octal), which has no defined printing character; when viewing a macro within most versions of TECO, "\$" characters are displayed as a substitution for <ESC>; this includes displaying <ESC> within the edit buffer using any of the usual viewing commands. This is why it is difficult to edit TECO macros other than within TECO itself. <ESC> is used as a terminator of a string or command in most instances; a macro must contain multiple <ESC> characters by nature. The situation is often complicated as TECO macros are known to dynamically create other TECO macros to be executed from the originating TECO macro program. As such, these created TECO macros will also contain multiple <ESC> characters.

Fortunately, TECO allows direct insertion of individual characters by character code. The sequence 27I\$ is often used for the purpose of inserting <ESC> into the edit buffer.

Note: Some programmers are more comfortable with octal values; by executing the Control-O command, all further numeric input will be evaluated in octal radix until a Control-D command is executed to return the input radix to decimal. In that context, the sequence 32I\$ is often used to achieve the equivalent result. Future releases of STRIP.TEC may use octal radix notation given the number of special case character codes processed.

Similarity of different commands.

The Insert (I) command, when passed a numerical argument (and terminated by an escape character), creates a single character in the edit buffer with the code of the numerical argument. The numerical form of the "I" command requires a trailing escape character to distinguish this from the text form of the "I" command which (potentially) allows multiple insertions of text strings.

Note: Specifics of the text form of the "I" command could be version-dependent. STRIP.TEC avoids command extensions where possible. The text form of the "I" command (as typically used when editing files manually) inserts the entirety of an indefinite text string into the edit buffer when the string terminator is reached. However, it might be used to insert one text character (as opposed to a numerically evaluated character).

Various TECO commands have an analogous duality of functionality determined by exact usage. In some instances, the contents of TECO macros must be carefully studied to determine the exact form of the command being used.

The description of the entire STRIP.TEC file follows. Further below, each section is explained in detail as to what its purpose is. Please note that line breaks (<CR>/<LF>) are meaningless within TECO except when those characters would be part of a string passed to the textual form of the "I" command. Thus, line breaks were added for readability.

Known Limitations.

Due to the way OS/8 creates quirky output with the PTP: handler, it is possible for trailing <FF> characters to occasionally still be present in the file after stripping. If necessary, the file may be stripped again to remove the trailing <FF> character. Since the last revision of STRIP.TEC, this problem has not been observed. Please report any problems with the latest release to the author.

STRIP.TEC Contents.

```
EZ#128EZ$HXQHKGQ0JIEB$ZJ27I$HXBHKMB
Y0J<N^X$;-1A&127U1-D
Q1-26"NONOTEND$'."EONOCARS$'
-3A-12"NONOFF$'-.3UZQZJ!NOFF!0,.P0,.K!NOCHARS!EFEX$
!NOTEND!Q1"EOP18CH1$'Q1-127"EOIGNORE$'OFFCHK$
!PL8CH1!0A+1"NOIGNORE$'-7A-12"NOIGNORE$'-7DOIGNORE$
!FFCHK!Q1-12"NONOTFF$'0A&127-12"EOIGNORE$'QF"NOPASTBEG$
'-.11"L0,.DOIGNORE$'!PASTBEG!%F
'.USQH"EOIGNORE$'0A+1"EOIGNORE$'2A+1"NOPL8CH2$
'2DOIGNORE$!PL8CH2!6A+1"NOFRSTCHK$
'6DOIGNORE$!FRSTCHK!QP"NOLOOP1$
'%P$.-4"NOLOOP1$'-4DOIGNORE$
!LOOP1!0A&127"NOLFTTEST$'DOLOOP1$
!LFTTEST!0A&127-10"NONOTFF$'C$
!LOOP2!0A&127"NOENDTEST$'DOLOOP2$
!ENDTEST!0A+1"NONOTE0F$'QS"EONOC$'0,QSP0,QSK
!NOCH!EFEX$!NOTE0F!QSJ0PUTIN$
!NOTFF!Q1-10"NOPUTIN$'-1A-13"EOPUTIN$'13I$
!PUTIN!Q1I$Q1-13"NONOTCR$'0A&127-10"N10I$'
!NOTCR!%H!IGNORE!>EFEX$
```

Detailed explanation.

Code Fragment	Description
Line #1:	EZ#128EZ\$HXQHKGQ0JIEB\$ZJ27I\$HXBHKMB
EZ#128EZ\$	EZ is a version-dependent internal register of TECOC which contains various control functions which can affect overall operation. The value already contained in the EZ register is inclusive .OR. (IOR) with the constant 128 then written back to the EZ register. By forcing this specific bit to the on state, buffer reads limited by <FF> characters are inhibited. Other TECO implementations could handle this situation differently or not at all; the logic of the STRIP.TEC macro depends on this operational behavior to search for <FF> characters.
HXQ	All of the (wHole (H)) text in the edit buffer is copied (X) into Q-register Q. The referenced text is the passed complete pathname argument, which is the expansion of the STRIPFILE environment variable such as: %PDP-8_DRIVE%\PEPS\Folders\Working_Files\PTPOUT.TXT which in turn contains the environment variable PDP-8_DRIVE. This further reduces to something of the form: P:\PEPS\Folders\Working_Files\PTPOUT.TXT This string is placed in the edit buffer by invoking TECOC in this manner. Note: The STRIPFILE environment variable is generally derived from the %1 argument to the batch file initiating the TECOC command. Command-line arguments to invoke TECO are extremely system-dependent; this method is valid for the MS-DOS and WINDOWS versions of TECOC.
HK	Kills the entire (wHole (H)) passed edit buffer, since it is now saved in Q-register Q.
GQ	Brings the passed string back into the edit buffer. Note: This method is used to allow additional file commands perhaps needed in a future version of the program.
0JIEB\$	Jump to the beginning of the buffer (0J) and insert "EB" there. The Edit Backup (EB) command, when fully formed, will be executed as a TECO macro.
ZJ27I\$	A trailing escape character is required to complete the macro command when it is executed. The buffer pointer is set to point to the end of the edit buffer then an escape character is placed into the edit buffer to complete the command.
HXB	The entire (wHole (H)) edit buffer contents, which is something of the form: EBP:\PEPS\Folders\Working_Files\PTPOUT.TXT\$ is copied (X) into Q-register B.
HK	Kills the entire (wHole (H)) passed edit buffer, since it is now saved in Q-register B.
MB	Q-register B's string contents is executed using a Macro B (MB) command. This sets up the passed file for editing.

Line #2: Y0J<N^X\$;-1A&127U1-D

Y The Yank (Y) command reads some portion of the input file into the edit buffer. Since <FF> checking is inhibited, only overall buffer size determines the exact quantity read in at any point. TECO implementations vary as to exact edit buffer size; a number of additional locations are reserved in case the edit buffer grows larger. Typical usage of the STRIP.TEC program will generally maintain the overall file size; certain specific operations might tend to raise or lower the overall size slightly depending on specific file contents. Most character processing consists of merely clearing the high-order (mark parity) bit. As such, the overall character count tends to remain relatively constant.

0J Reset the buffer pointer (J) to the beginning of the edit buffer (0). Most TECO implementations will do this automatically after a Yank operation. TECO programmers often include this to clarify the scope of an iteration loop that follows.

< Because of the leading "<" character, what follows up to a matching ">" character is executed in an infinite loop; various methods exist to break out of the loop, such as a failing search operation. This is the primary operational loop for most of the program.

N^X\$; A Nonstop search is performed; all characters processed will be forced into the output buffer and (eventually) written out. In this case, the search is a special-case operation indicated by the single character Control-X. This means the search argument is any single character of any value. The only way the search can fail is when the search returns a special value indicating there is no further input; this can also be detected by alternate means.

Note: This condition is not searchable in the ordinary sense. It is actually the status of the special condition of the buffer pointer position at the actual end of the input, not an actual character.

When any form of search inside of iteration brackets fails and a trailing ";" character exists, the normal error abort is inhibited; the iteration loop ends and execution continues past the ">" character matching the "<" character in effect when the failing search was attempted. It is quite typical to find this coding when encountering the end of any input file.

Note: In STRIP.TEC there is only one general iteration loop. TECO allows any number of nested iteration loops as needed, including the "breaking out" function if needed.

Assuming the search succeeded, the buffer pointer is now just past the matching character. Should a character be inserted, the buffer pointer will be placed just past the newly inserted character. In some instances the matching character will be deleted (causing the buffer pointer to be placed just past the previous character, if any).

Note: Character insertion or deletion after a match is only performed in certain specific cases.

Control-X as used here is the single character Control-X (ASCII code 24 decimal, 30 octal); in other contexts, the two-character sequence "^" and "X" (or "x") is also allowed.

However, all search arguments in TECO must be stated literally.

Note: Most TECO implementations echo the Control-X character as "^X"; when viewing macros such as STRIP.TEC, the correct usage must be confirmed as display operations are generally anomalous in this regard.

-1A&127U1-D

When the "A" command is used with a leading numerical argument, the enumerated value is used as an offset from the buffer pointer to locate a particular character in the buffer. In this instance the numeric value of the previous character (-1) is returned (which is the character that matched the search as described above). This value is then masked against the constant 127 (177 octal) via a logical .AND. operation which preserves the low-order seven bits while removing the high-order bit. The filtered value is then stored (U) in numeric Q-register 1 for further testing; the original character (located one position before (-) the current buffer pointer value) is then Deleted (D) from the edit buffer. (Later in the program, the test character will generally be inserted into the edit buffer unless special handling is required.)

Line #3:

Q1-26"NONOTEND\$'."EONCHARS\$'

Q1-26"NONOTEND\$'

If the test character is not Control-Z (ASCII code 26 decimal, 32 octal), transfer control (using the gOtO (O) command) to the portion of the program labeled !NOTEND!. If the character is Control-Z, execution continues inline to finalize the file in the next code section.

Note: The host operating system may have different conventions regarding the end of file state from the operating system that created the file; this is why end of file detection is also carried out elsewhere in the program. ASCII text files created for/by OS/8 or the P?S/8 SHELL generally conform to this convention. Extraneous characters may exist in the file past the Control-Z sentinel character. All such characters will be ignored.

Note: The P?S/8 SHELL is an optional feature that will be implemented in the next full release, sometime after Version 9A is completed. Consult various P?S/8 documents for more information.

In extremely rare situations, defective OS/8 utilities may create files that end on the last character of the last file record without using a sentinel Control-Z character; this situation is correctly handled by the STRIP.TEC program. However, it may be necessary to remedy this situation within OS/8 to add a proper Control-Z sentinel character. MS-DOS and Windows do not support this convention internally; exact byte counts are used to account for the proper file length. P?S/8 conversion programs to/from OS/8 use safeguards against this problem; OS/8 PIP operating in ASCII text mode should be used to correct this problem as required. Kermit-12 for OS/8 implements internal defensive measures to prevent this problem by making this situation impossible to occur. If need be, an entire extra record containing only Control-Z characters would be appended onto the file ending point.

."EONCHARS\$'

If the edit buffer pointer is zero, there were no characters in the buffer when the Control-Z sentinel character was detected. This is not a likely situation; however, this is a portion of the "belts and suspenders" approach taken within this program. Should this condition apply, transfer control to the portion of the program labeled *!NOCHARS!* bypassing all commands beyond this point in the program.

Line #4:

-3A-12"NONOFF\$' .-3UZQZJ!NOFF!0, .P0, .K!NOCHARS!EFEX\$

-3A-12"NONOFF\$'

The implementation of the standard OS/8 high-speed reader/punch coresident device handlers is quite poor. The reader handler will generally drop the last character; this is problematic when the contents of the data stream are not surrounded by leader/trailer codes.

Note: It is not clear if the implemented design logic actually applies to properly adjusted hardware; it may have merely been used as a defensive measure applied by an ignorant programmer not aware of the maintenance issues of the PC04/PC05 reader/punch devices. This was especially true in the case of DEC in-house development systems known to have had shoddy maintenance procedures. In essence, program development was often performed on aberrant hardware inconsistent with user systems nominally configured with the same devices.

In the PEPS environment, the notion of a malformed character does not apply; as such, it is necessary to obtain every character. Unfortunately, the handler design is such that a double-buffered approach is taken that will predictably ignore the final character.

To remedy this problem, OS/8 PIP may be used to restore the final character (generally a <LF>) or be certain the OS/8 program that will process the file ignores <LF> characters. (OS/8 PAL8 delimits source lines with <CR> and ignores <LF>.)

When using P?S/8 OS8CON to convert OS/8 input to P?S/8 output files, this case is properly handled; when converting P?S/8 text files to OS/8 text files, the result is always properly and completely formed with <CR> and <LF> on every line and a Control-Z character is always provided as a sentinel character to signify proper end-of-file.

The OS/8 high-speed punch handler situation is far worse: A mis-feature intended only for the Typeset-8 group was foisted on the general OS/8 user community despite it having no constructive purpose.

The mis-feature is the deliberate insertion of otherwise extraneous characters to attempt to format long paper tapes to be placed into the reader stations of offline Teletype Model ASR-35 terminals, which were generally available in all in-house Typeset-8 installations. A more sensible arrangement would include a Typeset-8 or other program to properly handle the particulars of this terminal; instead, the paper tape is greatly lengthened because of badly implemented attempts to directly format and print the output.

while this conceptual approach is quite bad, the implementation is even worse: Proper handling of the <HT> character on this device depends on exact carriage position. Depending on the particulars, one or perhaps two fill characters must be placed into the data stream directly to be self-contained as apparently was the case. A simple formula **should** have been used to determine the proper fill character count; instead a single <NUL> (or perhaps <RO>) character is always inserted just past the <HT> character. While this method does work in nearly half of the applicable cases that apply, this causes the output to be needlessly slowed down where the fill character is not required, and when two fill characters are needed, it is inadequate, leading to character smear on the terminal paper as the carriage is still in motion when the character is printed.

Various filler character strategies (both adequate and inadequate) are used for the supported control characters, which includes <FF>, <VT> and <HT>. Quite pointlessly, different fill characters are used in some cases different from others. The approach used is sometimes simple-minded; the worst-case quantity of fill characters is used instead of using a program method that keeps track of where on the printed page the current carriage position is located. (Had that better method been used, the typical count of fill characters would be far less.)

The STRIP.TEC macro is designed to eliminate every instance of undesirable output; as such, a properly formatted output file is created suitable for further processing. As such, there are certain quirky cases that must be addressed in this section of the program.

While some of the associated overhead has already been removed elsewhere, a specific case has been observed where a <FF> followed by <CR>/<LF> is in the buffer just before the Control-Z sentinel character. Since one of the goals of the program is to remove trailing <FF> characters, this section was added to explicitly remove this undesirable character sequence.

If the last three characters just before the Control-Z sentinel character are <FF> <CR> <LF> they will be removed. Checking three characters before the Control-Z sentinel character, compare that character to <FF>. If not a match, proceed to the next section of the program (labeled **!NOFF!**).

Note: This simplified method is adequate for this purpose. The reason is that it is not possible for there to be a line that does not have both a <CR> and a <LF> at this point in the filtering process. As such, the two characters after the <FF> are already completely determined and all three characters can be removed with impunity.

.-3UZQZJ!

If the tested character matches <FF>, store it's position in Q register Z; then change the edit buffer pointer to match this position effectively ignoring the three undesirable characters. (The label **!NOFF!** follows.)

!NOFF!0,.P0,.K

All characters in the edit buffer to this point (0,.) are output using the Punch (P) command then deleted using the Kill (K) command. These operations are skipped if the edit buffer is empty. (The label **!NOCHARS!** follows.)

!NOCHARS!EFEX\$

The output file is closed using the End File (EF) command; TECO EXits (EX) to the operating system.

Line #5: **!NOTEND!Q1"EOPL8CH1\$'Q1-127"EOIGNORE\$'OFFCHK\$**

!NOTEND!

The rest of the program processes the test character checking for various special cases. All additional operations are within the iteration brackets ("**<**" and "**>**") and will be performed repeatedly. Each iteration starts with the Non-stop search for any existent character until failing due to lack of further input. (Note: All processing for any iteration ends at the label **!IGNORE!** whether control is transferred there explicitly or execution falls through from the prior command.)

Q1"EOPL8CH1\$

If the test character is a **<NUL>** (ASCII code 0), transfer control to the portion of the program labeled **!PL8CH1!**. Files created by OS/8 PAL8 must be checked for a potential quirk associated with **<NUL>** characters near the end of the file. The **<NUL>** character will be removed; additional output filtering may be performed.

Note: Surprisingly, many OS/8 text files processed through the PTP: handler contain significant quantities of **<NUL>** characters typically in large groups. Similar special cases are known to exist which are handled elsewhere in this program.

'Q1-127"EOIGNORE\$

If the latest character is a **<RO>** (ASCII code 127 decimal, 177 octal]), it will be removed by transferring control to the portion of the program labeled **!IGNORE!**; the latest character will not be placed into the output file. Various files are known to contain extraneous **<RO>** characters for a variety of reasons.

Note: Virtually all P?S/8 device 66 lineprinter output starts with a single **<RO>** character. This was implemented based on misleading information suggesting this as the proper technique to initialize the printer interface. Future releases of P?S/8 will output a leading **<NUL>** character to enhance compatibility with certain quirky printers where a graphic character (such as a heart) is printed when **<RO>** is transmitted. Proper implementation will allow a one-word patch to restore the historic value of **<RO>** if desired.

'OFFCHK\$

Transfer control to the portion of the program labeled **!FFCHK!** to handle potential **<FF>** character situations.

Line #6: **!PL8CH1!0A+1"NOIGNORE\$'-7A-12"NOIGNORE\$'-7DOIGNORE\$**

!PL8CH1!

A **<NUL>** character was detected in the output file. If this is the last character in the file, determine if the file was created by PAL8 without the /C (CREF) option set. In this specific instance, PAL8 creates quirky output of several blank lines after the last **<FF>** character with the first line incompletely formed.

This actually represents two different problems: a) The file should end with a **<FF>** character (ignoring trailing **<NUL>** characters which will be removed), b) The blank lines are not formed correctly; one consists of merely a **<LF>** character.

When the /C option is passed to PAL8, a chain to CREF.SV will be performed; the final output file will correctly end with a **<FF>** character (ignoring trailing **<NUL>** characters which will be removed). The blank lines are still present in the output file; STRIP.TEC includes programming to correct defectively formed blank lines. This will lead to a deliberate blank page in the output consistent with the original behavior of PAL8 when used with physical printers.

Note: Post-processing of the output file may disallow blank pages; disposition of blank pages is beyond the scope of this program.

Since a design goal of STRIP.TEC is to remove the last <FF> character, the entire sequence will be removed.

Note: The sequence of characters removed consists of <FF> <LF> <CR> <LF> <CR> <LF>; the last character in the file is a <NUL> character which will also be removed.

Other than this quirky specific case, no properly formed file should be modified. Another portion of the program deals with the a similar problem when <NUL> characters are not involved should the file become available in the host environment by an exceptional process that avoids the PTP: handler artifacts.

0A+1"NOIGNORE\$

If the code returned for the next character following (0A) the detected <NUL> character is -1, transfer control to the portion of the program labeled *!IGNORE!* to remove the latest <NUL> character. This portion of the program is concerned only with a <NUL> character that is at the end of the file.

'-7A-12"NOIGNORE\$

If the character before the previous five characters is not a <FF> (ASCII code 12 decimal, 14 octal), transfer control to the portion of the program labeled *!IGNORE!* to remove the last <NUL> character in the file; the file does not have the quirky special case present.

'-7DOIGNORE\$

Delete the <FF> and all of the other characters through the end of the file; transfer control to the portion of the program labeled *!IGNORE!* to continue the iteration loop.

Note: The next iteration will break out of the loop when the nonstop search fails to complete this operation.

Line #7:

!FFCHK!Q1-12"NONOTFF\$'0A&127-12"EOIGNORE\$'QF"NOPASTBEG\$

!FFCHK!

This portion of the program checks for <FF> characters. This is a pivotal test as anomalies may be present either just before or just after a <FF> character. It is theorized most of these problems stem from an incomplete understanding of how to properly format logical print pages within various programs. Conforming text files separate pages with a single <FF> character; unfortunately, several anomalies may be present.

Q1-12"NONOTFF\$'

If the test character is not a <FF> (ASCII code 12 decimal, 14 octal), transfer control to the portion of the program labeled *!NOTFF!*.

0A&127-12"EOIGNORE\$'

The test character is a <FF>. If the next character is a <FF> (ASCII code 12 decimal, 14 octal), transfer control to the portion of the program labeled *!IGNORE!* (to ignore the test character). The next iteration will handle the situation such that a string of adjacent <FF> characters will all be ignored. Note: The latest character must be .AND. 127 (177 octal) since the latest character could have the parity bit set.

QF"NOPASTBEG\$

Q-register F is tested to determine if this section has been executed previously. If so, transfer control to the portion of the program labeled *!PASTBEG!* to avoid repeat processing. (The next section must be used only once.)

Line #8: '-.11"L0,.DOIGNORE\$'!PASTBEG!%F

'-.11"L0,.DOIGNORE\$' Since the <FF> character match is the first one encountered in the file, it is possible there are a few characters just before the <FF> which is generally the first character in the file (assuming it is at the beginning of the file at all). However, certain programs output small groups of test characters before the <FF> to ensure the viability of the lineprinter interface. If this is the case, all of these characters will be deleted.

Transfer control to the portion of the program labeled *!IGNORE!* to continue processing; initial <FF> characters are removed.

!PASTBEG!%F Control is transferred to this portion of the program to ensure the once-only section is not used again. Q-register F is incremented accordingly.

Line #9: '.USQH"EOIGNORE\$'0A+1"EOIGNORE\$'2A+1"NOPL8CH2\$

'.US The current edit buffer position (.) is stored (U) in numeric Q-register S. Certain operations may be performed on characters located past the current position; after these operations complete, the buffer pointer can be restored using the contents of Q-register S that was stored at this earlier point in the program.

QH"EOIGNORE\$' As a special accommodation for output files created by PAL8 and related OS/8 utilities, if the first character in the file is a <FF> character (after first removing all leading <NUL> and <RO> characters), it will be removed. Additional <NUL> or <RO> characters may follow; removal is accomplished elsewhere in the program.

Notes:

When output files from PAL8 and related OS/8 utilities are processed by the device 66 LPT: handler, additional leading <FF> character(s) may be inserted by the handler. The program logic is designed to eliminate any number of leading <FF> characters until a printing character is processed (including <CR> or <LF>).

P?S/8 PAL and related utilities (from the circa 1978 release of P?S/8 Version 8T) do not conform to this convention; the first <FF> and any prior <CR> and <LF> characters will be removed in another section of the program.

Note: This program depends on the known behavior of TECOC for MS-DOS and Windows to initialize numeric Q-register storage to zero. This behavior is generally implementation-specific. If the TECO implementation used supports errors on uninitialized variables, the program should be modified to initialize variables using commands such as OUH.

0A+1"EOIGNORE\$ STRIP.TEC attempts to remove trailing <FF> characters if possible; text files are handled differently in the SIMH host environment from classic direct printing techniques on physical printing hardware; post-processing is generally applied before final printing (or further conversion for publishing purposes). As such, steps must be taken to avoid trailing blank pages.

TECO implements safeguards associated with the end of file condition. Attempts to access additional characters will result in an error abort. To avoid this situation, if the code returned for the next character following (0A) (the <FF> character) is -1, the end of file has been reached. Control will be transferred to the portion of the program labeled *!IGNORE!* to avoid processing the trailing <FF> character; the next iteration will cause the search to fail and break out of the iteration loop to allow proper file closing and exit.

Note: In the unlikely event a file contains alternate quirky output anomalies not quite identical to the condition checked for here, the test for this condition will not detect the actual problem present; should this occur, a viable remedy is to run the STRIP.TEC program a second (and perhaps even a third) time. On reprocessed files, existing tests should successfully remove the trailing <FF>.) (Note: Before this feature was added to STRIP.TEC, removal of this specific quirk of PAL8 required a two-pass approach.

If the end of file has not been reached, program flow continues.

'2A+1"NOPL8CH2\$

If the buffer pointer is not at the end of file, check if the end of file is after exactly two additional characters (2A). (Note: The most likely case of extraneous output is <CR> followed by <LF> after the (otherwise) trailing <FF> character.) In general, this condition does not apply. Transfer control to the portion of the program labeled *!PL8CH2!*.

Line #10:

'2DOIGNORE\$!PL8CH2!6A+1"NOFRSTCHK\$

'2DOIGNORE\$

Remove the two characters past the (otherwise) trailing <FF> character (2D) . Then transfer control to the portion of the program labeled *!IGNORE!*; the trailing <FF> character will be removed.

Note: Future releases of STRIP.TEC may reorganize this program section.

!PL8CH2!

This portion of the program also checks for the quirky output of PAL8 when the /C (CREF) option is not used. The same logic is used with the exception of the <NUL> character consideration. Files that are present in the host environment transmitted by means other than the PTP: handler may still have the same quirky output.

Note: Files processed by prior versions of STRIP.TEC may require processing by this (newer) release.

6A+1"NOFRSTCHK\$

If the end of file is not after exactly six additional characters, transfer control to the portion of the program labeled *!FRSTCHK!*.

If the end of file is detected after exactly six additional characters, program flow continues on the next line following.

Line #11:

'6DOIGNORE\$!FRSTCHK!QP"NOLOOP1\$

'6DOIGNORE\$

Delete the six additional characters. Transfer control to the portion of the program labeled *!IGNORE!*; the trailing <FF> character will be removed.

!FRSTCHK! This is the section of the program labeled *!FRSTCHK!*. When the program continues here, the portion of the edit buffer being processed is not at the end of the file. As such, initial <FF> checking (when a <FF> is not the first character in the file) can be checked at this point.

QP"NOLOOP1\$ If this is not the first <FF> encountered (because the contents of Q-register P is non-zero), transfer control to the portion of the program labeled *!LOOP1!*. Only the initial detected <FF> character is considered in this section.

Note: This test is required for files where the first <FF> is not the first character in the file (but only a few additional characters precede the first <FF> character which should be removed).

Line #12: '%P\$.-4"NOLOOP1\$'-4DOIGNORE\$

'%P\$.-4"NOLOOP1\$ Increment (%) the contents of Q-register P to prevent additional initial <FF> testing in later program iterations.

Note: Certain printing files created by the circa 1978 release of P?S/8 (Version 8T) are known to start the output with a small number (four) of introductory characters prior to the first <FF> character. If this is not the case, transfer control to the portion of the program labeled *!LOOP1!*.

'-4DOIGNORE\$ If the introductory characters are present before the current edit buffer position, delete (D) the first four characters (-4). Transfer control to the portion of the program labeled *!IGNORE!* to remove the initial <FF> character.

Note: Newer releases of P?S/8 eliminated this problem; only output oriented to console terminals should have initial <CR>/<LF>-only lines before the first printing page (and only if the proper command switches are used to produce page-oriented output with tear-off lines).

When this portion of certain P?S/8 programs was designed, handling of physical lineprinters was not well documented. Later releases of P?S/8 generally do not exhibit this quirk in P?S/8 standard system programs (but may have alternate quirks handled elsewhere in this program). Certain user programs may exist that copied the older method; STRIP.TEC will correctly filter the output of all such programs by the logic implemented in this section.

Line #13: !LOOP1!0A&127"NOLFTEST\$'DOLOOP1\$

!LOOP1! In certain OS/8 files, sequences of up to nine <NUL> characters followed by a <LF> character are placed after every <FF> character, all of which must be removed. The program iteratively loops back to the portion of the program labeled *!LOOP1!* until all <NUL> characters after the <FF> character are removed.

0A&127"NOLFTEST\$' If the next character after a <FF> character is not a <NUL> character, transfer control to the portion of the program labeled *!LFTEST!*.

Note: All tests of this form must mask the high-order bit using the constant 127 (octal 177) because all characters not yet tested might have the mark parity bit set.

DOLOOP1\$ If the next character currently after a <FF> character is a <NUL> character, delete the character; transfer control back to the portion of the program labeled *!LOOP1!* to test for additional <NUL> characters.

Line #14: **!LFTEST!0A&127-10"NONOTFF\$'C\$**

!LFTEST! After all <NUL> characters after a <FF> character are removed, the next character is likely a <LF> character; test for this now.

0A&127-10"NONOTFF\$

If the character after all <NUL> characters after a <FF> character is not a <LF> (ASCII code 10 decimal, 12 octal), transfer control to the portion of the program labeled *!NOTFF!*.

Note: This applies if the file differs from the explicitly checked character sequence found in certain files created within OS/8.

'C\$ If the character after all <NUL> characters after a <FF> character is a <LF> character, advance the buffer pointer (C) to preserve the <LF> character.

Line #15: **!LOOP2!0A&127"NOENDTEST\$'DOLOOP2\$**

!LOOP2! In certain files passed through the OS/8 PTP: handler, a long sequence of trailing <NUL> characters may follow the last <FF> character followed by nine <NUL> characters followed by <LF>. All of these additional <NUL> characters will also be removed.

0A&127"NOENDTEST\$ If the character after the <LF> character is not a <NUL> character, transfer control to the portion of the program labeled *!ENDTEST!*.

'DOLOOP2\$ If the next character currently after the <LF> character is a <NUL> character, delete this character; transfer control to the portion of the program labeled *!LOOP2!* to test for additional <NUL> characters.

Line #16: **!ENDTEST!0A+1"NONOTEOF\$'QS"EONCH\$'0,QSP0,QSK**

!ENDTEST! An extremely pragmatic sequence known to be contained within certain OS/8 files processed by the PTP: handler must be located at the end of the file. If this is not the case, all further character processing will be handled conventionally.

0A+1"NONOTEOF\$ If the code returned for the next character following (0A) the latest <NUL> character is -1, the end of file has been reached. If this is not the case, transfer control to the portion of the program labeled *!NOTEOF!*.

'QS"EONCH\$' Q-register S contains the buffer pointer value where the trailing <FF> character was located before the now-validated trailing sequence of additional characters. If the contents of Q-register S is zero, control is transferred to the portion of the program labeled *!NOCH!* (because the edit buffer does not contain any characters to be processed into the output file).

0,QSP0,QSK All characters from the beginning of the buffer through the position indicated in Q-register S will be output using the Punch (P) command. All characters from the beginning of the buffer through the position indicated in Q-register S will then be deleted using the Kill (K) command. Program flow continues at the label *!NOCH!* which follows.

Line #17: **!NOCH!EFEX\$!NOTEOF!QSJOPUTIN\$**

!NOCH!EFEX\$ All retained characters in the edit buffer (if any) have been processed. Close the file with an End File (EF) command. TECO EXits (EX) to the operating system. All characters (within the pragmatic sequence specially checked for) were removed.

!NOTEOF!QSJOPUTIN\$ If the pragmatic sequence of characters expected after the <FF> did not occur, control is transferred here. The edit buffer pointer is restored (J) to the position saved earlier in Q-register S. Control is then transferred to the portion of the program labeled **!PUTIN!** to handle the latest <FF> character routinely.

Line #18: !NOTFF!Q1-10"NOPUTIN\$'-1A-13"EOPUTIN\$'13I\$

!NOTFF! Control is transferred to this portion of the program if either the latest character is not a <FF> character or the pragmatic sequence possibly present after a <FF> character in certain OS/8 files was not detected.

Q1-10"NOPUTIN\$ If the latest character is not a <LF> character, control is transferred to the portion of the program labeled **!PUTIN!**.

'-1A-13"EOPUTIN\$ If the latest character is a <LF> character, check if the previous character (-1A) is a <CR> character (ASCII code 13 decimal, 15 octal). If this is the case, transfer control to the portion of the program labeled **!PUTIN!**.

Note: It is not necessary to mask this value against the constant 127 (octal 177) because the program has already removed the high-order bit of all previous characters; all newly inserted characters do not have the high-order bit set.

'13I\$ If the latest character is a <LF> character and the previous character is not a <CR> character, insert a <CR> character before the <LF> character. Certain PDP-8-sourced files fail to obey the <CR> followed by <LF> convention; this is especially likely before or after a <FF> character; the host operating system requires lines to strictly end with <CR> followed by <LF>.

Note: Files originating from unix-compatible systems may require this <CR> correction at the end of every line. Program flow continues at the label **!PUTIN!** which follows.

Line #19: !PUTIN!Q1I\$Q1-13"NONOTCR\$'0A&127-10"N10I\$'

!PUTIN!Q1I\$ Control is transferred to this portion of the program to insert the latest character with the high-order bit removed. Q-register I contains the (possibly) corrected value which is inserted into the edit buffer at the current character position.

Q1-13"NONOTCR\$ If the latest character is not a <CR> character, control is transferred to the portion of the program labeled **!NOTCR!**.

'0A&127-10"N10I\$' If the latest character is a <CR> character, test the next character (0A) masking the high-order bit (&127); if the next character is not a <LF> character, insert a <LF> character after the <CR> character. Certain PDP-8 files (occasionally) fail to obey the <CR> followed by <LF> convention (although there are generally more occurrences of failing to have a <CR> before a <LF>). This situation can arise if the file originates from various Apple or other products. Certain P?S/8 programs from the circa 1978 Version 8T release have similar quirks; newer releases will correct these problems (if they haven't already been fixed). Program flow continues at the label **!NOTCR!** which follows.

Line #20: !NOTCR!%H!IGNORE!>EFEX\$\$

!NOTCR!%H Q-register H is incremented (%) to inhibit further special handling of potential initial <FF> characters. Other than the characters associated with infrequent special handling, Q-register H will generally contain the running character count of the file as it is being processed. Program flow continues at the label *!IGNORE!* which follows.

!IGNORE!> Any direct transfer of control to this point will ignore the latest character. All special-case routines that process characters also come here to finish the latest iteration loop indicated by the ">" character.

EFEX When the non-stop search fails, control is transferred here to the first command outside of the iteration brackets. The file is closed with the End File (EF) command; TECO Exits (EX) to the operating system.

\$\$ Since the entire macro string ends with two successive <ESC> characters, this ends the macro input as TECO is initialized. Macro execution starts with the first character of the file.

Note: The exact process of macro initialization is extremely version-dependent. As such, it is good practice to avoid the use of the textual Q-registers X, Y and Z; certain TECO implementations (other than TECOC) pass macro string arguments using methods that resemble the following:

HXZMZ

In essence, the entire macro is treated as a text string in the editing buffer as TECO starts up; all characters are then placed into Q-register Z (or X or Y depending on version) then the Macro Z (MZ) command is executed to start the contents of Q-register Z. In some versions of TECO (including TECOC), this is accomplished using a "hidden" Q-register. As such, the full complement of 36 Q-registers is always available.

[end-of-file]